



CERTIFICAT UNIVERSITAIRE – INTERNET DES OBJETS (IoT)

Projet Final : UML et Développement Orienté Objet

Cours : Conception orientée objets (UML)

Enseignant : Pr. Hamza JAMIRI

Étudiante : Wadline PIERRESSAINT

Année : 2025 – 2026

Date de remise : 05 / 02 / 2026

1. Introduction

Dans ce projet, nous modélisons un système IoT de supervision des salles de classe, dont le rôle est d'assurer confort, sécurité et maîtrise énergétique en surveillant en continu température, présence, fumée, etc. À l'aide d'UML et de la programmation orientée objet, nous identifions les acteurs et les principales entités du système, décrivons leurs interactions au moyen de plusieurs diagrammes, puis réalisons une petite simulation pour illustrer la réaction du système à différents événements (détection de fumée, dépassement de seuil de température). L'objectif est de proposer un modèle clair, cohérent et extensible, qui puisse servir de base à une implémentation concrète ultérieure.

2. Acteurs internes et externes

En UML, un acteur est une entité qui interagit avec le système pour atteindre un objectif. On distingue :

- Acteurs externes : entités hors du logiciel (personnes ou systèmes tiers) qui envoient des requêtes au système ou reçoivent des informations.
- Acteurs internes : composants du système (modules logiciels, objets principaux) qui collaborent pour réaliser ces requêtes.

Dans notre projet, les acteurs sont donc répartis ainsi :

2.1 Acteurs externes

- **Administrateur**
Utilisateur responsable de la configuration du système. Il crée et met à jour les salles, associe les capteurs et actionneurs, définit les règles de fonctionnement (seuils, actions automatiques) et consulte l'historique des alertes.
- **Technicien**
Utilisateur chargé de l'exploitation au quotidien. Il visualise l'état des salles (mesures, alertes en cours), traite les alertes signalées par le système et peut envoyer des commandes manuelles aux actionneurs (allumage de la lampe, activation de la ventilation, etc.).

Tous les utilisateurs humains sont des acteurs externes et doivent d'abord s'authentifier pour garantir la traçabilité des opérations.

2.2 Acteurs internes (composants logiciels)

- **Capteurs**
Objets logiciels représentant les capteurs physiques (température, humidité, présence, fumée). Ils produisent périodiquement des mesures transmises au système.
- **Actionneurs**
Objets représentant les équipements pilotables, tels que la lampe intelligente et le système de ventilation. Ils changent d'état soit à la suite d'une règle automatique, soit sur commande du technicien.
- **Système de supervision IoT**

Composant central qui :

- enregistre les mesures reçues,
- applique les règles de fonctionnement,
- crée les alertes en cas d'anomalie,
- commande les actionneurs,
- gère les utilisateurs et leur authentification.

Ainsi, Administrateur et Technicien sont les deux acteurs externes, tandis que Capteurs, Actionneurs et le Système de supervision IoT constituent les principaux acteurs internes de notre système.

3. Présentation des diagrammes UML

3.1 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation présente une vue globale des fonctionnalités offertes par le système du point de vue des utilisateurs externes.

On y voit notamment que :

- l'administrateur peut s'authentifier, gérer les salles, installer les équipements et définir les règles de fonctionnement ;
- le technicien s'authentifie également, consulte l'état des salles, traite les alertes et envoie des commandes manuelles aux actionneurs.

Ce diagramme permet de vérifier rapidement que tous les besoins fonctionnels exprimés dans l'énoncé sont bien pris en compte.



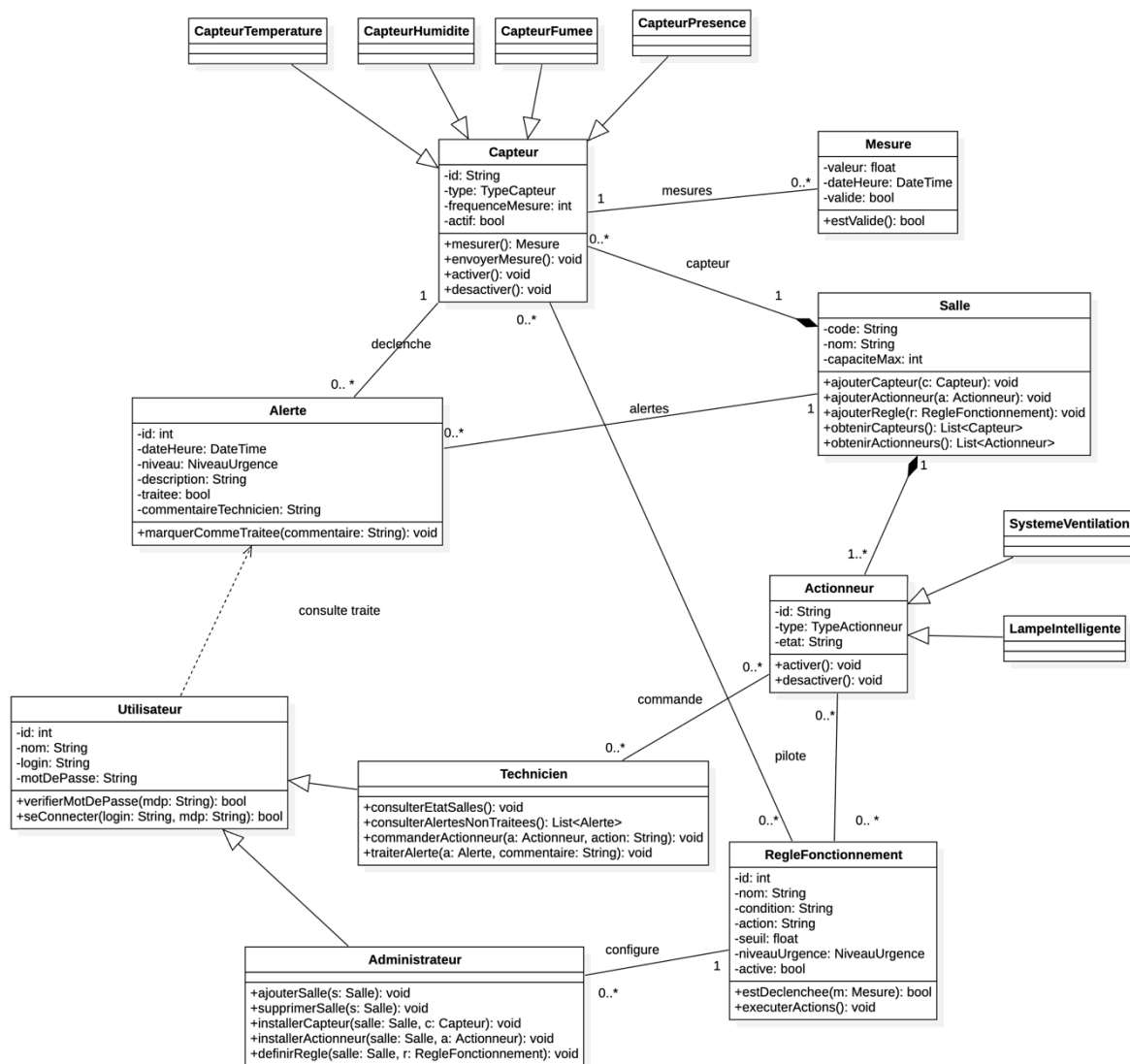
3.2 Diagramme de classes

Le diagramme de classes décrit la structure statique du système.

- La classe Salle regroupe les capteurs, les actionneurs et les règles de fonctionnement associés à un local particulier.
- Les classes Capteur et Actionneur sont spécialisées en plusieurs types (capteur de température, capteur de fumée, lampe intelligente, système de ventilation, etc.), ce qui facilite l'extension à de nouveaux équipements.

- La classe Mesure stocke la valeur relevée par un capteur à une date donnée, tandis que la classe Alerte mémorise les événements anormaux, avec leur niveau d'urgence et leur état de traitement.
- Les classes Administrateur et Technicien héritent d'une classe abstraite Utilisateur, ce qui factorise les attributs communs (identifiant, login, mot de passe) tout en distinguant les responsabilités de chaque rôle.
- Enfin, la classe RegleFonctionnement lie les conditions détectées sur les mesures (par exemple « température > seuil » ou « fumée détectée ») aux actions à effectuer sur les actionneurs.

Ce diagramme formalise le modèle conceptuel qui servira de base à l'implémentation orientée objet.



3.3 Diagrammes de séquence

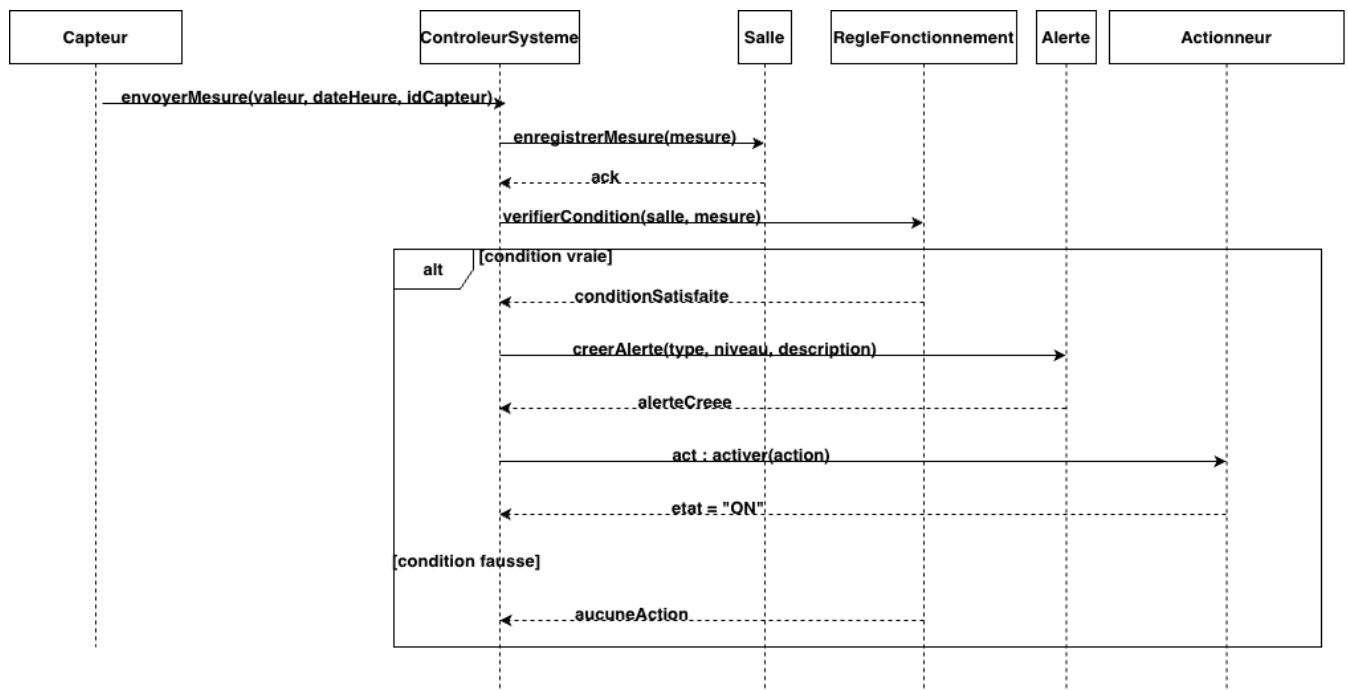
Les diagrammes de séquence décrivent l'enchaînement temporel des messages échangés entre objets pour réaliser un scénario particulier.

Un premier scénario décrit, par exemple, le traitement d'une alerte par un technicien :

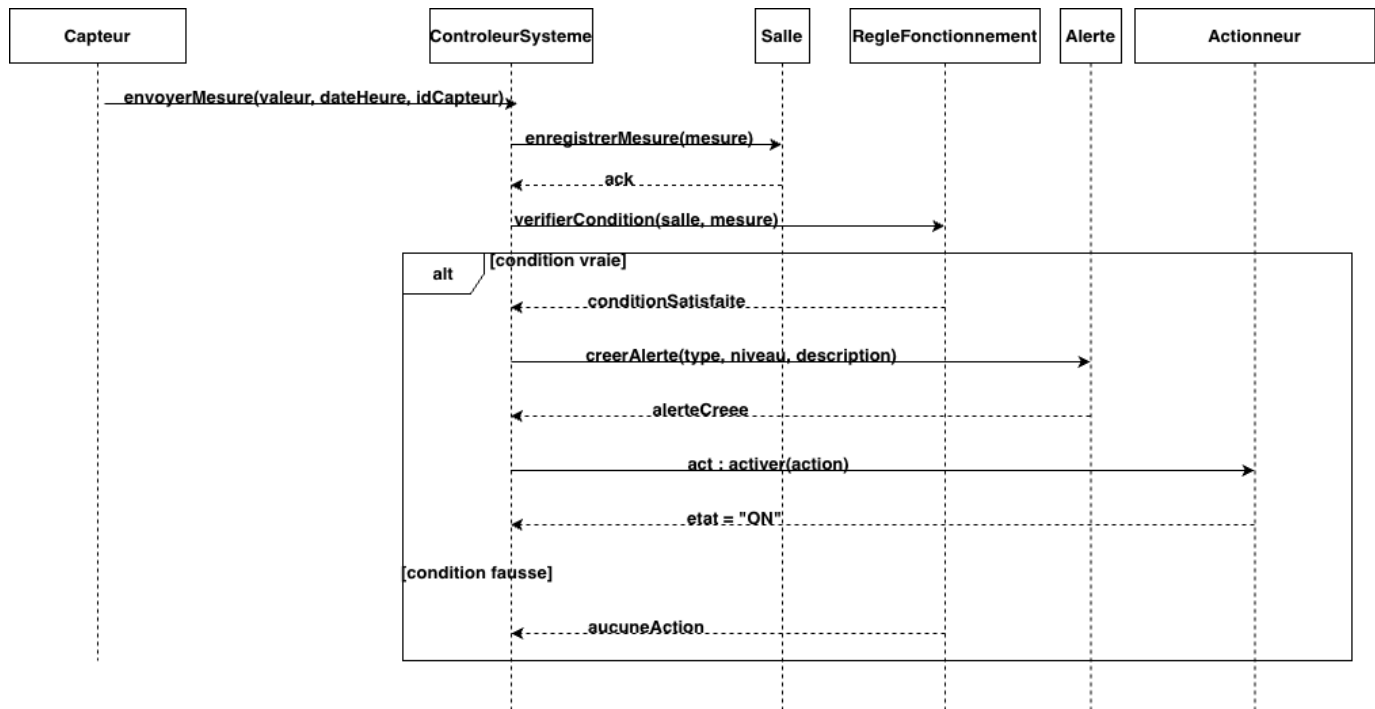
1. Le technicien demande à l'interface de consulter la liste des alertes non traitées.
2. L'interface interroge le contrôleur du système, qui récupère les alertes auprès du module de gestion des alertes.
3. Le technicien sélectionne ensuite une alerte dans la liste, et le contrôleur la marque comme traitée.
4. Si nécessaire, le technicien envoie une commande manuelle à un actionneur, qui change d'état et renvoie sa nouvelle valeur.

Ce type de diagramme met en évidence les responsabilités de chaque objet et la logique de collaboration entre la couche interface, la logique métier et les entités métiers.

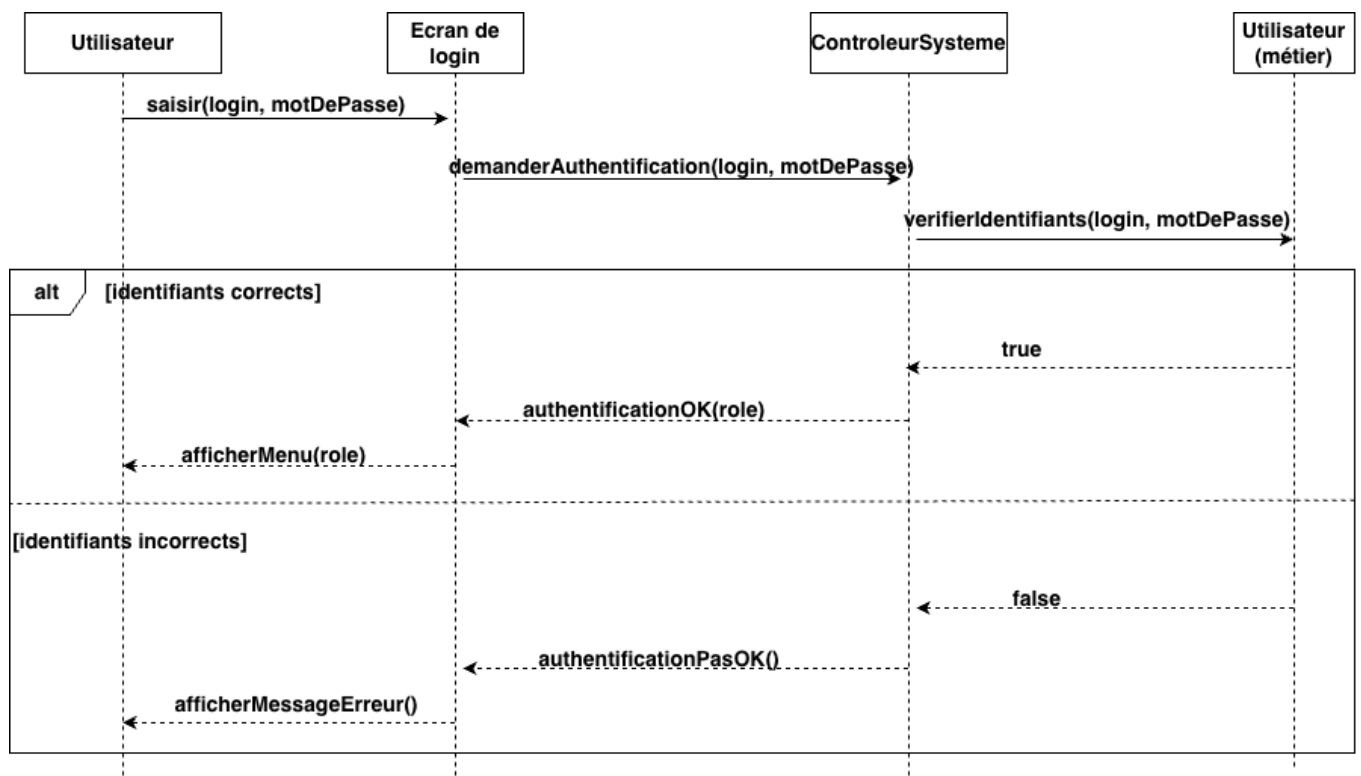
☐ Contrôle automatisé d'une salle



☐ Prise en charge d'une alerte par un technicien



□ Connexion d'un utilisateur au système



3.4 Diagrammes d'activité

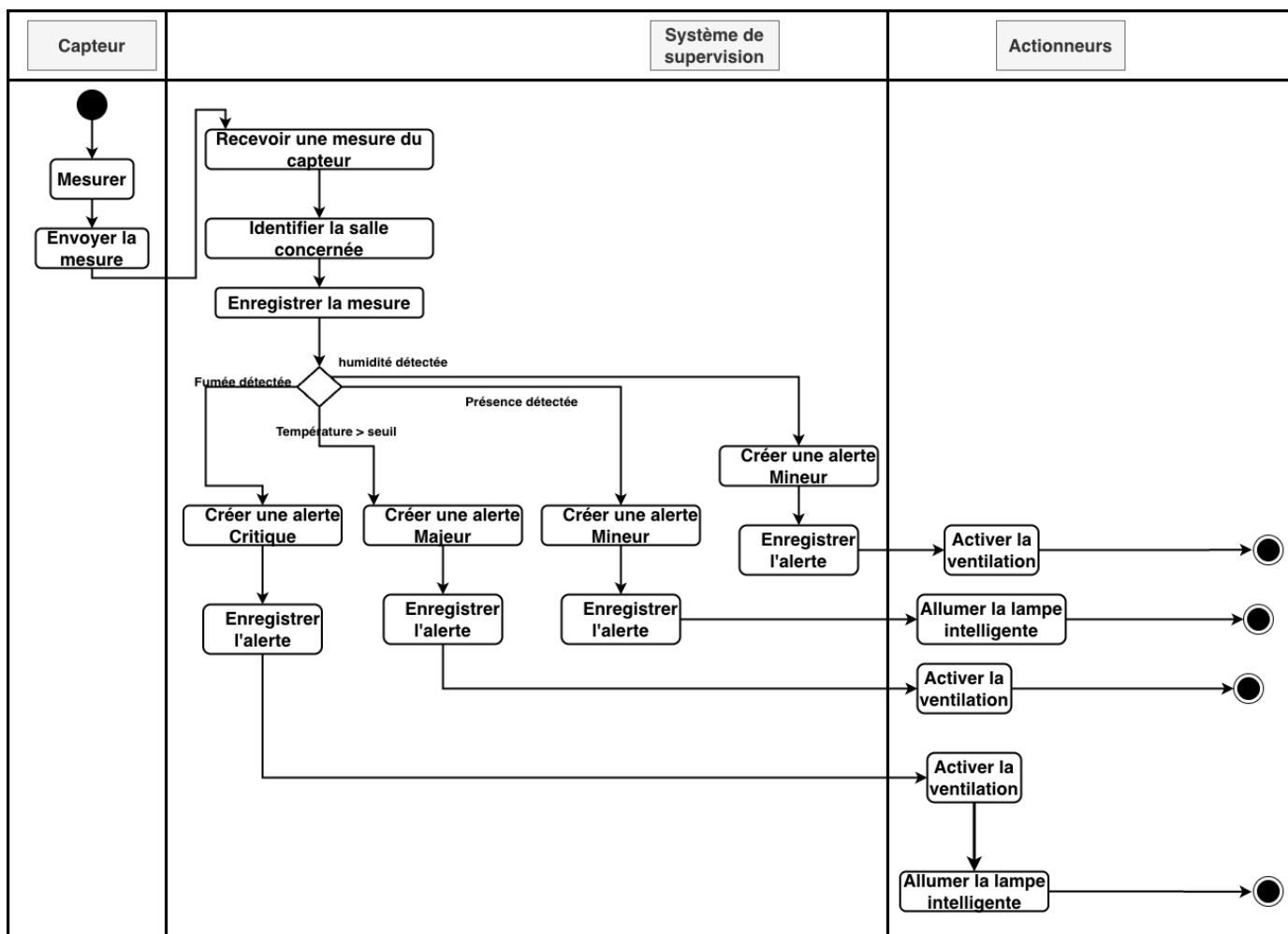
Les diagrammes d'activité représentent le flux de contrôle et les données à travers le système.

Notre diagramme d'activité décrit la surveillance automatique d'une salle :

- Dans la partition Capteur, une activité mesure la grandeur physique et envoie la mesure au système.
- Dans la partition Système, la mesure est enregistrée, puis les différentes règles associées à la salle sont évaluées.
- Si une condition est satisfaite, une activité crée une alerte et déclenche les actions correspondantes sur les actionneurs (mise en marche de la ventilation, allumage d'une lampe, etc.).
- Le flux d'activité se termine lorsque toutes les règles ont été vérifiées.

Ce type de diagramme aide à comprendre le comportement global et les différents chemins possibles en fonction des conditions.

□ Contrôle automatisé d'une salle



5. Simulation orientée objet

La partie logicielle a été réalisée sous la forme d'une application console écrite en C#, développée et exécutée avec Visual Studio Code et le SDK .NET. Le modèle objet reprend les principales entités définies en UML :

- Salle
- Capteur
- Mesure
- RegleFonctionnement
- Actionneur
- Alerte

ainsi que les utilisateurs :

- Administrateur
- Technicien.

Le système de supervision simule, à chaque cycle, l'envoi de mesures par les capteurs ; ces mesures sont comparées aux règles de fonctionnement de la salle et, en cas de dépassement de seuil, une alerte est créée et les actionneurs associés sont automatiquement activés.

Au démarrage, l'utilisateur s'authentifie :

si c'est un administrateur, il dispose d'un menu pour ajouter/supprimer des salles, installer des capteurs ou actionneurs et définir de nouvelles règles.

```
=== Menu Administrateur ===
1. Ajouter une salle
2. Supprimer une salle
3. Installer des équipements (capteurs / actionneurs)
4. Définir une règle de fonctionnement
5. Afficher l'état des salles
0. Quitter
Choix : █
```

S'il s'agit d'un technicien, il peut lancer la simulation, consulter l'état des salles, voir la liste des alertes non traitées et envoyer des commandes manuelles aux actionneurs.

```
=== Menu Technicien ===
1. Lancer une simulation (capteurs -> règles -> alertes -> actionneurs)
2. Consulter l'état des salles
3. Consulter les alertes non traitées
4. Commander un actionneur
0. Quitter
Choix : 0
```

Cette simulation illustre donc concrètement le comportement dynamique du système conçu en UML.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading;
```

```

namespace SalleIoT
{
    #region Enums et classes métier

    public enum TypeCapteur
    {
        Temperature,
        Humidite,
        Presence,
        Fumee
    }

    public enum NiveauUrgence
    {
        Mineur,
        Majeur,
        Critique
    }

    public enum TypeActionneur
    {
        LampeIntelligente,
        SystemeVentilation
    }

    public class Mesure
    {
        public DateTime DateHeure { get; set; }
        public float Valeur { get; set; }
        public bool Valide { get; set; } = true;

        public override string ToString()
        {
            return $"{DateHeure:HH:mm:ss} -> {Valeur}";
        }
    }

    public class Capteur
    {
        public string Id { get; set; }
        public TypeCapteur Type { get; set; }
        public int FrequenceMesureSecondes { get; set; } = 60;

        public Mesure GenererMesureAleatoire()
        {
            // Simulation simple de valeur selon le type
            var rnd = new Random();
            float valeur = Type switch
            {
                TypeCapteur.Temperature => rnd.Next(15, 40), // 15° à 40°
                TypeCapteur.Humidite => rnd.Next(30, 90),    // 30% à 90%
            }
        }
    }
}

```

```

        TypeCapteur.Presence => rnd.Next(0, 2),           // 0 ou 1
        TypeCapteur.Fumee => rnd.Next(0, 2),             // 0 ou 1
        _ => 0
    };

    return new Measure
    {
        DateHeure = DateTime.Now,
        Valeur = valeur,
        Valide = true
    };
}

}

public class Actionneur
{
    public string Id { get; set; }
    public TypeActionneur Type { get; set; }
    public bool Etat { get; private set; } // true = ON, false = OFF

    public void Activer()
    {
        Etat = true;
        Console.WriteLine($"    -> Actionneur {Id} ({Type}) ACTIVÉ");
    }

    public void Desactiver()
    {
        Etat = false;
        Console.WriteLine($"    -> Actionneur {Id} ({Type}) DÉACTIVÉ");
    }

    public override string ToString()
    {
        return $"{Id} ({Type}) - Etat : {(Etat ? "ON" : "OFF")}";
    }
}

public class RegleFonctionnement
{
    public int Id { get; set; }
    public string Nom { get; set; }
    public TypeCapteur CapteurCible { get; set; }
    public float Seuil { get; set; }
    public string ConditionTexte { get; set; }
    public NiveauUrgence Niveau { get; set; }
    public List<Actionneur> ActionneursCibles { get; set; } = new();

    /// <summary>
    /// Vérifie si la mesure déclenche la règle.
    /// Ici : si valeur >= seuil.

```

```

    /// </summary>
    public bool EstDeclencheePar(Mesure mesure)
    {
        return mesure.Valeur >= Seuil;
    }

    public override string ToString()
    {
        return $"[R{Id}] {Nom} - Capteur: {CapteurCible}, Seuil: {Seuil},
Niveau: {Niveau}";
    }
}

public class Alerte
{
    public int Id { get; set; }
    public string Description { get; set; }
    public NiveauUrgence Niveau { get; set; }
    public DateTime DateHeure { get; set; }
    public bool Traitee { get; set; } = false;
    public string CommentaireTechnicien { get; set; }

    public override string ToString()
    {
        return $"[A{Id}] {DateHeure:HH:mm:ss} - {Niveau} - {Description}" +
            (Traitee ? " (traitée)" : " (non traitée)");
    }
}

public class Salle
{
    public string Code { get; set; }
    public string Nom { get; set; }
    public int CapaciteMax { get; set; }

    public List<Capteur> Capteurs { get; set; } = new();
    public List<Actionneur> Actionneurs { get; set; } = new();
    public List<RegleFonctionnement> Regles { get; set; } = new();
}

public abstract class Utilisateur
{
    public string Login { get; set; }
    public string MotDePasse { get; set; }
    public string Nom { get; set; }

    public bool VerifierMotDePasse(string mdp) => MotDePasse == mdp;
}

public class Administrateur : Utilisateur
{

```

```

}

public class Technicien : Utilisateur
{
}

#endregion

#region Système de supervision

public class SystemeSupervision
{
    public List<Salle> Salles { get; } = new();
    public List<Alerte> Alertes { get; } = new();
    private int _prochainIdAlerte = 1;
    private int _prochainIdRegle = 1;

    #region Méthodes "capteurs -> règles -> alertes -> actionneurs"

    /// <summary>
    /// Simule n cycles de mesures (1 cycle = "1 minute").
    /// </summary>
    public void SimulerMesures(int nombreCycles)
    {
        Console.WriteLine("\n=== Simulation des mesures (1 cycle = 1 minute
simulée) ===\n");

        for (int cycle = 1; cycle <= nombreCycles; cycle++)
        {
            Console.WriteLine($"--- Cycle {cycle} ---");

            foreach (var salle in Salles)
            {
                foreach (var capteur in salle.Capteurs)
                {
                    var mesure = capteur.GenererMesureAleatoire();
                    Console.WriteLine(
                        $"Salle {salle.Code} | Capteur {capteur.Type} ->
{mesure.Valeur}");

                    TraiterMesure(salle, capteur, mesure);
                }
            }

            // On attend 1 seconde pour simuler 1 minute
            Thread.Sleep(1000);
        }
    }

    private void TraiterMesure(Salle salle, Capteur capteur, Mesure mesure)
    {

```

```

// Pour chaque règle de la salle qui cible ce type de capteur
var reglesCibles = salle.Regles
    .Where(r => r.CapteurCible == capteur.Type)
    .ToList();

foreach (var regle in reglesCibles)
{
    if (regle.EstDeclencheePar(mesure))
    {
        Console.WriteLine($"    -> Règle déclenchée : {regle.Nom}");
        var alerte = CreerAlerte(regle, salle, capteur, mesure);
        DeclencherActions(regle, alerte);
    }
}

private Alerte CreerAlerte(RegleFonctionnement regle, Salle salle,
                           Capteur capteur, Mesure mesure)
{
    var alerte = new Alerte
    {
        Id = _prochainIdAlerte++,
        DateHeure = DateTime.Now,
        Niveau = regle.Niveau,
        Description = $"Salle {salle.Code} - {capteur.Type}={mesure.Valeur}

" +
                           $"(Règle: {regle.Nom})"
    };
    Alertes.Add(alerte);

    Console.WriteLine($"    -> ALERTE CRÉÉE : {alerte}");

    return alerte;
}

private void DeclencherActions(RegleFonctionnement regle, Alerte alerte)
{
    Console.WriteLine("    -> Exécution des actions associées...");
    foreach (var act in regle.ActionneursCibles)
    {
        // Exemple : pour l'instant, on active l'actionneur
        act.Activer();
    }
}

#endregion

#region Fonctions administrateur

public void AjouterSalle()
{

```

```

        Console.WriteLine("Code de la salle : ");
        var code = Console.ReadLine();
        Console.WriteLine("Nom de la salle : ");
        var nom = Console.ReadLine();
        Console.WriteLine("Capacité max : ");
        int.TryParse(Console.ReadLine(), out var cap);

        var salle = new Salle
        {
            Code = code,
            Nom = nom,
            CapaciteMax = cap
        };
        Salles.Add(salle);

        Console.WriteLine("Salle ajoutée.");
    }

    public void SupprimerSalle()
    {
        Console.WriteLine("Code de la salle à supprimer : ");
        var code = Console.ReadLine();
        var salle = Salles.FirstOrDefault(s => s.Code == code);
        if (salle == null)
        {
            Console.WriteLine("Salle introuvable.");
            return;
        }

        Salles.Remove(salle);
        Console.WriteLine("Salle supprimée.");
    }

    public void InstallerEquipements()
    {
        var salle = ChoisirSalle();
        if (salle == null) return;

        Console.WriteLine("1. Ajouter un capteur");
        Console.WriteLine("2. Ajouter un actionneur");
        Console.WriteLine("Choix : ");
        var choix = Console.ReadLine();

        if (choix == "1")
        {
            Console.WriteLine("Type de capteur  
(0=Température,1=Humidité,2=Présence,3=Fumée) : ");
            int.TryParse(Console.ReadLine(), out var t);
            var capteur = new Capteur
            {
                Id = $"C{salle.Capteurs.Count + 1}",
            }
        }
    }

```

```

        Type = (TypeCapteur)t
    };
    salle.Capteurs.Add(capteur);
    Console.WriteLine("Capteur installé.");
}
else if (choix == "2")
{
    Console.WriteLine("Type d'actionneur (0=Lampe,1=Ventilation) : ");
    int.TryParse(Console.ReadLine(), out var t);
    var act = new Actionneur
    {
        Id = $"A{salle.Actionneurs.Count + 1}",
        Type = (TypeActionneur)t
    };
    salle.Actionneurs.Add(act);
    Console.WriteLine("Actionneur installé.");
}
}

public void DefinirRegle()
{
    var salle = ChoisirSalle();
    if (salle == null) return;

    Console.WriteLine("Type de capteur concerné  
(0=Température,1=Humidité,2=Présence,3=Fumée) : ");
    int.TryParse(Console.ReadLine(), out var tCapteur);

    Console.WriteLine("Nom de la règle : ");
    var nom = Console.ReadLine();

    Console.WriteLine("Seuil de déclenchement : ");
    float.TryParse(Console.ReadLine(), out var seuil);

    Console.WriteLine("Niveau d'urgence (0=Mineur,1=Majeur,2=Critique) : ");
    int.TryParse(Console.ReadLine(), out var niv);

    var regle = new RegleFonctionnement
    {
        Id = _prochainIdRegle++,
        Nom = nom,
        CapteurCible = (TypeCapteur)tCapteur,
        Seuil = seuil,
        Niveau = (NiveauUrgence)niv,
        ConditionTexte = $"valeur >= {seuil}"
    };

    // Associer tous les actionneurs de la salle pour simplifier
    regle.ActionneursCibles.AddRange(salle.Actionneurs);
    salle.Regles.Add(regle);
}

```

```

        Console.WriteLine("Règle ajoutée :");
        Console.WriteLine(regle);
    }

#endregion

#region Fonctions technicien

public void AfficherEtatSalles()
{
    Console.WriteLine("\n=== État des salles ===");
    foreach (var s in Salles)
    {
        Console.WriteLine($"Salle {s.Code} - {s.Nom} (Capacité
{s.CapaciteMax})");
        Console.WriteLine("  Capteurs :");
        foreach (var c in s.Capteurs)
        {
            Console.WriteLine($"    - {c.Id} ({c.Type})");
        }

        Console.WriteLine("  Actionneurs :");
        foreach (var a in s.Actionneurs)
        {
            Console.WriteLine($"    - {a}");
        }

        Console.WriteLine("  Règles :");
        foreach (var r in s.Regles)
        {
            Console.WriteLine($"    - {r}");
        }
        Console.WriteLine();
    }
}

public void AfficherAlertesNonTraitees()
{
    Console.WriteLine("\n=== Alertes non traitées ===");
    var list = Alertes.Where(a => !a.Traitee).ToList();
    if (!list.Any())
    {
        Console.WriteLine("Aucune alerte.");
        return;
    }

    foreach (var a in list)
    {
        Console.WriteLine(a);
    }
}

```

```

}

public void CommanderActionneur()
{
    var salle = ChoisirSalle();
    if (salle == null) return;

    if (!salle.Actionneurs.Any())
    {
        Console.WriteLine("Aucun actionneur dans cette salle.");
        return;
    }

    Console.WriteLine("Actionneurs disponibles :");
    for (int i = 0; i < salle.Actionneurs.Count; i++)
    {
        Console.WriteLine($"{i + 1}. {salle.Actionneurs[i]}");
    }

    Console.Write("Choisissez un actionneur : ");
    int.TryParse(Console.ReadLine(), out var idx);
    if (idx < 1 || idx > salle.Actionneurs.Count)
    {
        Console.WriteLine("Choix invalide.");
        return;
    }

    var act = salle.Actionneurs[idx - 1];
    Console.Write("Commande (1=Activer, 2=Désactiver) : ");
    var cmd = Console.ReadLine();

    if (cmd == "1") act.Activer();
    else if (cmd == "2") act.Desactiver();
}

#endregion

private Salle ChoisirSalle()
{
    if (!Salles.Any())
    {
        Console.WriteLine("Aucune salle dans le système.");
        return null;
    }

    Console.WriteLine("Salles disponibles :");
    foreach (var s in Salles)
    {
        Console.WriteLine($"{s.Code} : {s.Nom}");
    }
}

```

```

        Console.WriteLine("Code de la salle : ");
        var code = Console.ReadLine();
        var salle = Salles.FirstOrDefault(s => s.Code == code);

        if (salle == null) Console.WriteLine("Salle introuvable.");
        return salle;
    }
}

#endregion

#region Programme principal (connexion + menus)

class Program
{
    static void Main()
    {
        var systeme = InitialiserSystemeDemo();

        // Utilisateurs de démo
        var admin = new Administrateur
        {
            Login = "admin",
            MotDePasse = "admin",
            Nom = "Administrateur"
        };

        var technicien = new Technicien
        {
            Login = "tech",
            MotDePasse = "tech",
            Nom = "Technicien"
        };

        Console.WriteLine("==== Système IoT de gestion des salles ====\\n");

        // Formulaire de connexion
        Utilisateur user = null;
        while (user == null)
        {
            Console.WriteLine("Login : ");
            var login = Console.ReadLine();
            Console.WriteLine("Mot de passe : ");
            var mdp = Console.ReadLine();

            if (login == admin.Login && admin.VerifierMotDePasse(mdp))
                user = admin;
            else if (login == technicien.Login &&
technicien.VerifierMotDePasse(mdp))
                user = technicien;
            else

```

```

        Console.WriteLine("Identifiants invalides.\n");
    }

    Console.WriteLine($"Bienvenue, {user.Nom} !\n");

    // Menus selon le rôle
    if (user is Administrateur)
        MenuAdministrateur(systeme);
    else if (user is Technicien)
        MenuTechnicien(systeme);

    Console.WriteLine("\nFin du programme.");
}

static SystemeSupervision InitialiserSystemeDemo()
{
    var systeme = new SystemeSupervision();

    // Une salle préconfigurée pour la démo
    var salle = new Salle
    {
        Code = "A101",
        Nom = "Salle de cours A101",
        CapaciteMax = 30
    };

    var capteurTemp = new Capteur { Id = "C1", Type =
TypeCapteur.Temperature };
    var capteurFumee = new Capteur { Id = "C2", Type = TypeCapteur.Fumee };

    var lampe = new Actionneur { Id = "A1", Type =
TypeActionneur.LampeIntelligente };
    var ventilo = new Actionneur { Id = "A2", Type =
TypeActionneur.SystemeVentilation };

    salle.Capteurs.Add(capteurTemp);
    salle.Capteurs.Add(capteurFumee);
    salle.Actionneurs.Add(lampe);
    salle.Actionneurs.Add(ventilo);

    // Règle : si température >= 30 -> alerte majeur + activer ventilation
    var regleTemp = new RegleFonctionnement
    {
        Id = 1,
        Nom = "Température élevée",
        CapteurCible = TypeCapteur.Temperature,
        Seuil = 30,
        Niveau = NiveauUrgence.Majeur,
        ConditionTexte = "température >= 30°C"
    };
    regleTemp.ActionneursCibles.Add(ventilo);
}

```

```

        // Règle : fumée détectée -> alerte critique + allumer lampe +
ventilation
        var regleFumee = new RegleFonctionnement
        {
            Id = 2,
            Nom = "Fumée détectée",
            CapteurCible = TypeCapteur.Fumee,
            Seuil = 1,
            Niveau = NiveauUrgence.Critique,
            ConditionTexte = "fumée détectée"
        };
        regleFumee.ActionneursCibles.Add(lampe);
        regleFumee.ActionneursCibles.Add(ventilo);

        salle.Regles.Add(regleTemp);
        salle.Regles.Add(regleFumee);

        systeme.Salles.Add(salle);

        return systeme;
    }

    static void MenuAdministrateur(SystemeSupervision systeme)
    {
        string choix;
        do
        {
            Console.WriteLine("\n=== Menu Administrateur ===");
            Console.WriteLine("1. Ajouter une salle");
            Console.WriteLine("2. Supprimer une salle");
            Console.WriteLine("3. Installer des équipements (capteurs /
actionneurs)");
            Console.WriteLine("4. Définir une règle de fonctionnement");
            Console.WriteLine("5. Afficher l'état des salles");
            Console.WriteLine("0. Quitter");
            Console.Write("Choix : ");
            choix = Console.ReadLine();

            switch (choix)
            {
                case "1": systeme.AjouterSalle(); break;
                case "2": systeme.SupprimerSalle(); break;
                case "3": systeme.InstallerEquipements(); break;
                case "4": systeme.DefinirRegle(); break;
                case "5": systeme.AfficherEtatSalles(); break;
            }
        } while (choix != "0");
    }

    static void MenuTechnicien(SystemeSupervision systeme)

```

```

{
    string choix;
    do
    {
        Console.WriteLine("\n=== Menu Technicien ===");
        Console.WriteLine("1. Lancer une simulation (capteurs -> règles ->
alertes -> actionneurs)");
        Console.WriteLine("2. Consulter l'état des salles");
        Console.WriteLine("3. Consulter les alertes non traitées");
        Console.WriteLine("4. Commander un actionneur");
        Console.WriteLine("0. Quitter");
        Console.Write("Choix : ");
        choix = Console.ReadLine();

        switch (choix)
        {
            case "1":
                Console.Write("Nombre de cycles de simulation : ");
                int.TryParse(Console.ReadLine(), out var n);
                if (n <= 0) n = 3;
                systeme.SimulerMesures(n);
                break;
            case "2":
                systeme.AfficherEtatSalles();
                break;
            case "3":
                systeme.AfficherAlertesNonTraitees();
                break;
            case "4":
                systeme.CommanderActionneur();
                break;
        }
    } while (choix != "0");
}

#endregion
}

```

6. Conclusion

Ce travail a permis de structurer la conception d'un système IoT de gestion de salles de classe en suivant une démarche orientée objet. Les diagrammes UML (cas d'utilisation, classes, séquences et activités) offrent des points de vue complémentaires : ils clarifient les besoins fonctionnels, la structure des données et le comportement du système face aux événements.

La simulation logicielle réalisée à partir de ces modèles montre que la conception est cohérente et relativement simple à faire évoluer. De nouvelles règles, de nouveaux types de capteurs ou d'actionneurs pourraient être intégrés sans remettre en cause l'architecture globale. UML joue ainsi pleinement son rôle de pont entre la spécification des besoins, la conception et une future implémentation industrielle du système IoT d'Al-Nour.